Facultatea de Matematică şi Informatică
Universitatea din Bucureşti



Programare orientată pe obiecte

- suport de curs -

Anca Dobrovăț

An universitar 2025 – 2026
Semestrul I
Seria 26
Curs 1

Generalităţi despre curs

1. Curs - vineri, orele 12 – 15

2. Laborator - in fiecare saptamana

3. Seminar - o data la 2 saptamani

4. Prezenta la curs/seminar: nu e obligatorie!

   Laborator – OBLIGATORIU

Să ne cunoaștem

Cine predă?

Curs: Anca Dobrovăț
- anca.dobrovat@fmi.unibuc.ro

Seminar: Anca Dobrovăț (261/262), Maxim Tiberiu (263/264)

Laboratoare:

- Marius Micluță – Câmpeanu (261, 264)
- Tiberiu Maxim (262, 263)

Agenda cursului

1. Regulamente UB si FMI

2. Utilitatea cursului de Programare Orientata pe Obiecte

3. Prezentarea disciplinei

4. Primul curs

Agenda cursului

1. Regulamente UB si FMI

2. Utilitatea cursului de Programare Orientata pe Obiecte

3. Prezentarea disciplinei

4. Primul curs

1. Regulamente UB si FMI

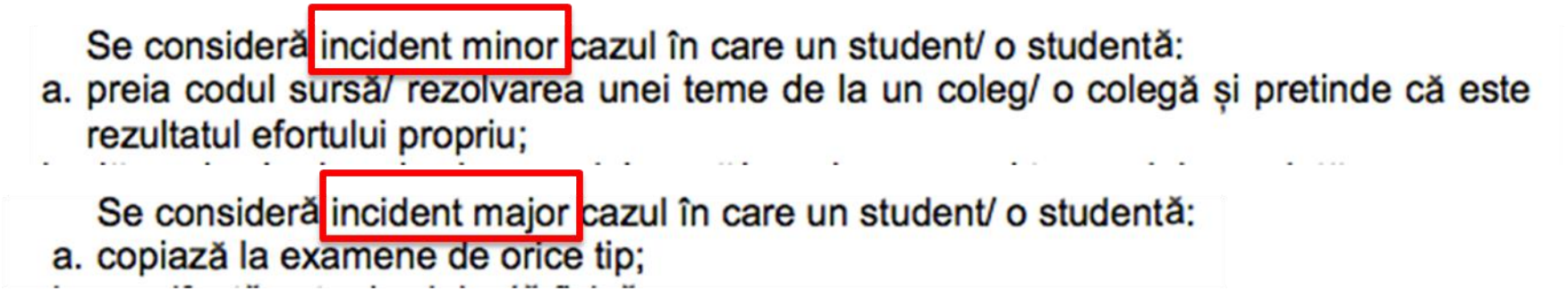
Lucruri bine de stiut de studenti:

• https://fmi.unibuc.ro/regulamente/

•regulament privind activitatea studenților la UB: https://unibuc.ro/wp-content/uploads/2024/04/Regulamentul-privind-activitatea-profesionala-a-studentilor-2024.pdf

• regulament de etică și profesionalism la FMI:

http://fmi.unibuc.ro/ro/pdf/2015/consiliu/Regulament_etica_FMI.pdf



Se consideră incident minor cazul în care un student/ o studentă:

a. preia codul sursă/ rezolvarea unei teme de la un coleg/ o colegă și pretinde că este rezultatul efortului propriu;

Se consideră incident major cazul în care un student/ o studentă:

a. copiază la examene de orice tip;

3 incidente minore = un incident major = exmatriculare

Agenda cursului

1. Regulamente UB si FMI

2. Utilitatea cursului de Programare Orientata pe Obiecte

3. Prezentarea disciplinei

4. Primul curs

2. Utilitatea cursului de Programare Orientata pe Obiecte

PYPL PopularitY of Programming Language

Captura din: http://pypl.github.io/PYPL

Worldwide, Oct 2025 :

Rank	Change	Language	Share	1-year trend
1		Python	28.97 %	-0.5 %
2		Java	13.94 %	-1.5 %
3	↑	C/C++	10.54 %	+3.6 %
4	↑↑↑↑↑↑↑	Objective-C	7.05 %	+4.5 %
5	↓↓	JavaScript	6.33 %	-1.7 %
6		R	5.27 %	+0.6 %
7	↓↓	C#	3.96 %	-2.5 %
8	↓	PHP	3.19 %	-0.8 %
9		Swift	2.93 %	+0.3 %

Worldwide, Sept 2024 :

Rank	Change	Language	Share	1-year trend
1		Python	29.66 %	+1.6 %
2		Java	15.64 %	-0.2 %
3		JavaScript	8.3 %	-1.0 %
4		C#	6.64 %	-0.1 %
5		C/C++	6.46 %	-0.2 %
6	↑	R	4.66 %	+0.2 %
7	↓	PHP	4.35 %	-0.5 %
8		TypeScript	2.96 %	-0.0 %
9		Swift	2.69 %	+0.0 %
10	↑	Rust	2.65 %	+0.6 %
11	↓	Objective-C	2.45 %	+0.2 %

Majoritatea pot fi considerate limbaje OO.

2. Utilitatea cursului de Programare Orientata pe Obiecte

Paradigme de programare → Stil fundamental de a programa

Dictează:

- **Cum se reprezintă datele problemei** (variabile, funcții, obiecte, fapte, constrângeri etc.)
- **Cum se prelucrează reprezentarea** (atribuiri, evaluări, fire de execuție, continuări, fluxuri etc.)
- **Favorizează un set de concepte si tehnici de programare**
- **Influențează felul în care sunt gândiți algoritmii de rezolvare a problemelor**
- **Limbaje – în general multiparadigmă (ex: Python – imperativ, funcțional, orientat pe obiecte)**

Agenda cursului

1. Regulamente UB si FMI
2. Utilitatea cursului de Programare Orientata pe Obiecte
3. Prezentarea disciplinei
    - 3.1 Obiectivele discipinei
    - 3.2 Programa cursului
    - 3.3 Bibliografie
    - 3.4 Regulament de notare si evaluare
4. Primul curs

3. Prezentarea disciplinei

3.1 Obiectivele disciplinei

- Curs de programare OO

- Oferă o **baza** de pornire pentru alte cursuri

- **Obiectivul general al disciplinei:**

Formarea unei imagini generale, preliminare, despre programarea orientată pe obiecte (POO).

- **Obiective specifice:**
  - •1. Înțelegerea fundamentelor paradigmei programarii orientate pe obiecte;
  - •2. Înțelegerea conceptelor de clasă, interfață, moștenire, polimorfism;
  - •3. Familiarizarea cu şabloanele de proiectare;
  - •4. Dezvoltarea de aplicații de complexitate medie respectând principiile de dezvoltare ale POO;
  - •5. Deprinderea cu noile facilităţi oferite de limbajul C++.

3. Prezentarea disciplinei

3.2 Programa cursului

1. Prezentarea disciplinei.
1.1 Principiile programării orientate pe obiecte.
1.2. Caracteristici.
1.3. Programa cursului, obiective, desfăşurare, examinare, bibliografie.

2. Recapitulare limbaj C (procedural) și introducerea în programarea orientată pe obiecte.
2.1 Funcții, transferul parametrilor, pointeri.
2.2 Deosebiri între C și C++.
2.3 Supradefinirea funcțiilor, Operații de intrare/ieșire, Tipul referință, Funcții în structuri.

3. Prezentarea disciplinei

3.2 Programa cursului

3. Proiectarea ascendenta a claselor. Incapsularea datelor in C++.
3.1 Conceptele de clasa și obiect. Structura unei clase.
3.2 Constructorii și destructorul unei clase.
3.3 Metode de acces la membrii unei clase, pointerul this. Modificatori de acces în C++.
3.4 Declararea și implementarea metodelor în clasă și în afara clasei.

4. Supraîncărcarea funcțiilor și operatorilor în C++.
4.1 Clase și funcții friend.
4.2 Supraîncărcarea funcțiilor.
4.3 Supraîncărcarea operatorilor cu funcții friend.
4.4 Supraîncărcarea operatorilor cu funcții membru.
4.5 Observații.

3. Prezentarea disciplinei

3.2 Programa cursului

5. Conversia datelor în C++.
5.1 Conversii între diferite tipuri de obiecte (operatorul cast, operatorul= și constructor de copiere).
5.2 Membrii constanți și statici ai unei clase in C++.
5.3 Modificatorul const, obiecte constante, pointeri constanți la obiecte și pointeri la obiecte constante.

6. Tratarea excepțiilor in C++.

7. Proiectarea descendenta a claselor. Mostenirea in C++.
7.1 Controlul accesului la clasa de bază.
7.2 Constructori, destructori şi moştenire.
7.3 Redefinirea membrilor unei clase de bază într-o clasa derivată.
7.4. Declaraţii de acces.

3. Prezentarea disciplinei

3.2 Programa cursului

8. Funcții virtuale în C++.
8.1 Parametrizarea metodelor (polimorfism la executie).
8.2 Funcții virtuale în C++. Clase abstracte.
8.3 Destructori virtuali.

9. Mostenirea multiplă şi virtuală în C++
9.1 Moştenirea din clase de bază multiple.
9.2 Exemple, observaţii.

10. Controlul tipului în timpul rulării programului în C++.
10.1 Mecanisme de tip RTTI (Run Time Type Identification).
10.2 Moştenire multiplă şi identificatori de tip (dynamic_cast, typeid).

3. Prezentarea disciplinei

3.2 Programa cursului

11. Parametrizarea datelor. Şabloane în C++. Clase generice
11.1 Funcţii şi clase Template: Definiţii, Exemple, Implementare.
11.2 Clase Template derivate.
11.3 Specializare.

12. Biblioteca Standard Template Library - STL
12.1 Containere, iteratori şi algoritmi.
12.2 Clasele string, set, map / multimap, list, vector, etc.

3. Prezentarea disciplinei

3.2 Programa cursului

13. Şabloane de proiectare (Design Pattern)
13.1 Definiţie şi clasificare.
13.2 Exemple de şabloane de proiectare (Singleton, Abstract Object Factory).

14. Recapitulare, concluzii, tratarea subiectelor de examen.

3. Prezentarea disciplinei

3.3 Bibliografie

1. Herbert Schildt. C++ manual complet. Ed.Teora, Bucuresti, 1997 (si urmatoarele).

2. Bruce Eckel. Thinking in C++ (2nd edition). Volume 1: Introduction to Standard C++. Prentice Hall, 2000.

3. Bruce Eckel, Chuck Allison. Thinking in C++ (2nd edition). Volume 2: Practical Programming. Prentice Hall, 2003.
(Obs: cartile se pot descarca in format electronic, gratuit si legal de la adresa *http://mindview.net/Books/TICPP/ThinkingInCPP2e.html*)

4. Bjarne Stroustrup: The C++ Programming Language, Adisson-Wesley, 3rd edition, 1997.

5. B.Stroustrup, The C++ Programming Language, 4th Edition, Addison-Wesley, May 2013

6. Erich Gamma, Richard Helm, Ralph Johnson, John Vlissides: Design Patterns. Elements of Reusable Object-Oriented Software. Addison-Wesley, 1995.

3. Prezentarea disciplinei

3.4 Regulament de notare şi evaluare

Curs: 3 ore pe săptămână

Laborator: 2 ore pe săptămână.

Seminar: 1 ora pe săptămână.

Disciplina: semestrul I, durata de desfășurare de 14 săptămâni.

Materia este de nivel elementar mediu şi se bazează pe cunoștințele de C++ anterior dobândite.

Limbajul de programare folosit la curs şi la laborator este **C++**.

3. Prezentarea disciplinei

3.4 Regulament de notare si evaluare

Programa disciplinei este împărțită în 14 cursuri.

Evaluarea studenților se face cumulativ prin:
• Laborator (proiecte) – nr de proiecte depinde de tutorii de laborator dar se trece prin mai multe concepte
•Test practic (colocviu)
•Test scris

Toate cele 3 probe de evaluare sunt obligatorii.

Condiții de promovare - minim **nota 5 la fiecare** parte de evaluare enunțată - mai sus se păstrează la oricare din eventualele examene restante ulteriore aferente acestui curs.

3. Prezentarea disciplinei

3.4 Regulament de notare si evaluare

Testul practic (Colocviu) - in saptamana 14

Pentru a intra in colocviu, trebuie ca nota de laborator sa fie minim 5!

- Consta dintr-un program care trebuie realizat individual intr-un timp limitat (1h30 - 2h) si va avea un nivel mediu.

- Notare: de la 1 la 10, nu neaparat intregi (pot exista pana la 3 puncte bonus).

Testul practic este obligatoriu.

Studentii care **nu obtin cel putin nota 5 la testul practic de laborator nu pot intra in examen** si vor trebui sa il dea din nou, inainte de prezentarea la restanta.

3. Prezentarea disciplinei

3.4 Regulament de notare si evaluare

Testul scris:

Consta din:

- 15 intrebari de teorie, grila (unul sau mai multe raspunsuri corecte, adevarat / fals), fiecare valorand 0,2p.
- 12 subiecte practice (să se decida dacă un algoritm dat este corect (caz în care să menţioneze rezultatul), sau este incorect (caz în care trebuie să se recunoască și să se rectifice eroarea)), fiecare valorand 0,5p.
- 1 punct din oficiu.

Studentii nu pot lua examenul decat daca obtin cel putin nota 5 la testul scris.

3. Prezentarea disciplinei

3.4 Regulament de notare si evaluare

Examenul se considera luat daca studentul respectiv a obtinut cel putin nota 5 la fiecare dintre cele 3 evaluari (activitatea practica din timpul semestrului, testul practic de laborator si testul scris).

In aceasta situatie, nota finala a fiecarui student se calculeaza ca medie ponderata intre notele obtinute la cele 3 evaluari, ponderile cu care cele 3 note intra in medie fiind:

- 25% - nota pe lucrarile practice (proiecte)
- 25% - nota la testul practic
- 50% - nota la testul scris

Seminar - maxim 1p care se adauga la nota de la testul scris, **daca si numai daca**, nota de la testul scris >=5.

3. Prezentarea disciplinei

Modificari

- Seminar: 1p bonus la nota de la examenul scris
- Prezenta la curs: 1p bonus la nota de la examenul scris pentru primii 25% dintre studenti KAHOOT
- bonusuri dupa ce se promoveaza examenul scris

3. Prezentarea disciplinei

Kahoot

- Se va defini un nume unic de forma 261popescu (unde popescu este numele de familie si 261 este grupa)

- Daca sunt mai multi studenti cu acelasi nume in grupa respectiva (adaugati si initiala / initialele prenumelui)

- 261popescup si 261popescupr

Agenda cursului

1. Regulamente UB si FMI
2. Utilitatea cursului de Programare Orientata pe Obiecte
3. Prezentarea disciplinei
4. Primul curs

4.1 Completări aduse de limbajul C++ faţă de limbajul C

4.2 Principiile programarii orientate pe obiecte

4. Curs 1

4.1 Completări aduse de limbajul C++ faţă de limbajul C

- Bjarne Stroustrup în 1979 la Bell Laboratories in Murray Hill, New Jersey
- 5 revizii: 1998 ANSI+ISO, 2003 (corrigendum), 2011 (C++11/0x), 2014, 2017 (C++ 17/1z)
- Următoarea plănuită în 2020 (C++2a)
- Versiunea 1998: Standard C++, C++98

4. Curs 1

4.1 Completări aduse de limbajul C++ faţă de limbajul C

- C++98: a definit standardul inițial, toate chestiunile de limbaj, STL
- C++03: bugfix o unică chestie nouă: value initialization
- C++11: initializer lists, rvalue references, moving constructors, lambda functions, final, constant null pointer, etc.
- C++14: generic lambdas, binary literals, auto, variable template, etc.

4. Curs 1

4.1 Completări aduse de limbajul C++ faţă de limbajul C

- C++17:

- If constexpr()
- Inline variables
- Nested namespace definitions
- Class template argument deduction
- Hexadecimal literals
- etc

typename is permitted for template template parameter declarations
(e.g.,
`template<template<typename> typename X> struct ...`)

4. Curs 1

4.1 Completări aduse de limbajul C++ faţă de limbajul C

- <iostream> (fără .h)
- using namespace std;
- cout, cin (fără &)
- // comentarii pe o linie
- declarare variabile
- Tipul de date bool
- se definesc true şi false (1 si 0);
- C99 nu îl definește ca bool ci ca _Bool (fără true/false)
- <stdbool.h> pentru compatibilitate

4. Curs 1

4.1 Completări aduse de limbajul C++ faţă de limbajul C

Intrări şi ieşiri

Obiectele **cin** şi **cout**, în plus faţă de funcţiile scanf şi printf din limbajul C.

Nu necesită specificarea formatelor.

// operator este o functie care are ca nume un simbol (sau mai multe simboluri)

```
int x,y,z;

cin >>x;  // operator>>(cin,x) : intoarce fluxul (prin referinta ) cin din care s-a extras data x

cin>>y>>z; // operator>>(operator >>(cin,y), z)

cout<<x; // operator<<(cout,x) : intoarce fluxul cout (prin referinta) in care s-a inserat x

cout<<y<<z; // operator<<(operator<<(cout,y),z)
```

4. Curs 1

4.1 Completări aduse de limbajul C++ faţă de limbajul C

```
#include <iostream>
using namespace std;
int main()
{
        int i;
        cout << "This is output.\n"; // this is a single line comment
        /* you can still use C style comments */
        // input a number using >>
        cout << "Enter a number: ";
        cin >> i;
        // now, output a number using <<
        cout << i << " squared is " << i*i << "\n";
        return 0;
}
```

4. Curs 1

4.1 Completări aduse de limbajul C++ faţă de limbajul C

```
#include <iostream>
using namespace std;
int main( )
{
        float f;
        char str[80];
        double d;
        cout << "Enter two floating point numbers: ";
        cin >> f >> d;
        cout << "Enter a string: ";
        cin >> str;
        cout << f << " " << d << " " << str;
        return 0;
}
```

citirea string-urilor se face pană la primul caracter alb
se poate face afișare folosind toate caracterele speciale \n, \t, etc.

4. Curs 1

4.1 Completări aduse de limbajul C++ faţă de limbajul C

Variabilele locale

```
/* Incorrect in C89. OK in C++. */
int f()
{
        int i;
        i = 10;
        int j; /* aici problema de compilare in C */
        j = i*2;
        return j;
}
```

4. Curs 1

4.1 Completări aduse de limbajul C++ faţă de limbajul C

Supraîncărcarea funcţiilor (un caz de *Polimorfism la compilare*)

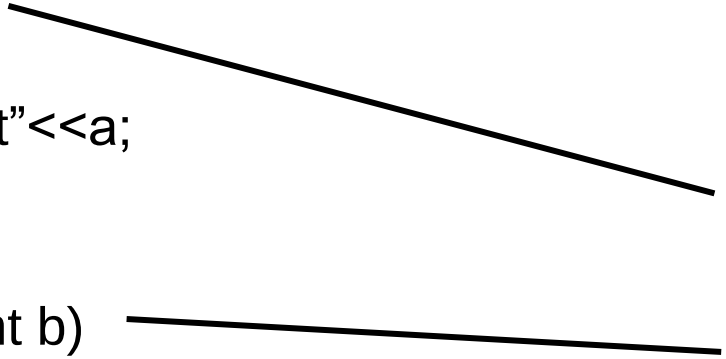
•Utilizarea mai multor funcţii care au acelaşi nume,
•Identificarea se face prin numărul de parametri şi tipul lor. **Tipul de întoarcere nu e suficient pentru a face diferența**

•simplicitate/corectitudine de cod

Exemplu:

```
void afis (int a)
{
        cout<<"int"<<a;
}

void afis (int a, int b)
{
        cout<<"char"<<a;
}
```



Apel:

afis (7);

afis (1,2);

4. Curs 1

4.1 Completări aduse de limbajul C++ faţă de limbajul C

```
#include <iostream>
using namespace std;

// abs is overloaded three ways
int abs(int i);
double abs(double d);
long abs(long l);

int main()
{
        cout << abs(-10) << "\n";
        cout << abs(-11.0) << "\n";
        cout << abs(-9L) << "\n";
        return 0;
}

int abs(int i)
{
        cout << "Using integer abs()\n";
        return i<0 ? -i : i;
}

double abs(double d)
{
        cout << "Using double abs()\n";
        return d<0.0 ? -d : d;
}

long abs(long l)
{
        cout << "Using long abs()\n";
        return l<0 ? -l : l;
}
```

```
Using integer abs()
10
Using double abs()
11
Using long abs()
9
```

4. Curs 1

4.1 Completări aduse de limbajul C++ faţă de limbajul C

Funcţii cu valori implicite

Într-o funcţie se pot declara valori implicite pentru unul sau mai mulţi parametri.

La apel se poate omite specificarea valorii pentru acei parametri formali care au declarate valori implicite.

Argumentele cu valori implicite trebuie să fie amplasate la sfârşitul listei.

```
void f (int a, int b = 12){  cout<<a<<" - "<<b<<endl;}

int main(){
  f(1);
  f(1,20);

  return 0;
}
```

4. Curs 1

4.1 Completări aduse de limbajul C++ faţă de limbajul C

Cand tipul returnat de o functie nu este declarat explicit, i se atribuie automat int.

Tipul trebuie cunoscut inainte de apel.

```
f (double x)
{
      return x;
}
```

Prototipul unei functii: permite declararea in afara si a numarului de parametri / tipul lor:

```
void f(int); // antet / prototip
int main() { cout<< f(50); }
void f( int x)
{
            // corp functie;
}
```

4. Curs 1

4.1 Completări aduse de limbajul C++ faţă de limbajul C

Funcţii cu valori implicite

Valorile implicite se specifică o singură dată în definiţie (de obicei în prototip).

```
#include <iostream>
using namespace std;

void f (int a, int b = 12); // prototip cu mentionarea valorii implicite pentru b

int main()
{
  f(1);
  f(1,20);
return 0;
}

void f (int a, int b) {  cout<<a<<" - "<<b<<endl; }
```

4. Curs 1

4.1 Completări aduse de limbajul C++ faţă de limbajul C

Alocare dinamica

```
int *pi;
 pi=new int;

 delete pi; // elibereaza  zona adresata de pi -o considera neocupata

   pi=new int(2);// aloca zona si initializeaza zona cu valoarea 2

   pi=new int[2]; // aloca un vector de 2 elemente  de tip intreg
   delete [ ] pi; //elibereaza intreg vectorul

     //-pentru new se foloseste delete
     //- pentru new [ ] se foloseste delete [ ]
```

4. Curs 1

4.1 Completări aduse de limbajul C++ faţă de limbajul C

Tipul referinta

O referinţă este, in esenta, un pointer implicit, care actioneaza ca un alt nume al unui obiect (variabila).

```
int i;
int *pi,j;
```

int & ri=i; //ri este alt nume pentru variabila i

```
pi=&i;  // pi este adresa variabilei i

*pi=3;   //in zona adresata de pi se afla valoarea 3
```

Pentru a putea fi folosită, o referinţă trebuie iniţializată in momentul declararii, devenind un alias (un alt nume) al obiectului cu care a fost iniţializată.

4. Curs 1

4.1 Completări aduse de limbajul C++ faţă de limbajul C

Tipul referinta

```
#include <iostream>
using namespace std;

int main()
{
  int a = 20;
  int& ref = a;
  cout<<a<<" "<<ref<<endl; // 20 20
  ref++;
  cout<<a<<" "<<ref<<endl; // 21 21
  return 0;
}
```

Obs: spre deosebire de pointeri care la incrementare trec la un alt obiect de acelasi tip, incrementarea referintei implica, de fapt, incrementarea valorii referite.

4. Curs 1

4.1 Completări aduse de limbajul C++ faţă de limbajul C

Tipul referinta

```
#include <iostream>
using namespace std;

int main()
{
  int a = 20;
  int& ref = a;
  cout<<a<<" "<<ref<<endl; // 20 20
  int b = 50;
  ref = b;
  ref--;
  cout<<a<<" "<<ref<<endl; // 49 49
  return 0;
}
```

Obs: in afara initializarii, nu puteti modifica obiectul spre care indica referinta.

4. Curs 1

4.1 Completări aduse de limbajul C++ faţă de limbajul C

Tipul referinta

• o referinta trebuie să fie initializata când este definita, dacă nu este membra a unei clase, un parametru de functie sau o valoare returnata;

•referintele nule sunt interzise intr-un program C++ valid.

•nu se poate obtine adresa unei referinte.

•nu se pot crea tablouri de referinte.

•nu se poate face referinta catre un camp de biti.

4. Curs 1

4.1 Completări aduse de limbajul C++ faţă de limbajul C

Transmiterea parametrilor

C

```
void f(int x){   x = x *2;}

void g(int *x){  *x = *x + 30;}

int main()
{
  int x = 10;
  f(x);
  printf("x = %d\n”,x);
  g(&x);
  printf("x = %d\n“,x);
  return 0;
}
```

C++

```
#include <iostream>
using namespace std;
void f(int x){   x = x *2;} //prin valoare
void g(int *x){  *x = *x + 30;} // prin pointer
void h(int &x){ x = x + 50;} //prin referinta

int main()
{
  int x = 10;
  f(x);
  cout<<"x = "<<x<<endl;
  g(&x);
  cout<<"x = "<<x<<endl;
  h(x);
  cout<<"x = "<<x<<endl;
  return 0;
}
```

4. Curs 1

4.1 Completări aduse de limbajul C++ faţă de limbajul C

Transmiterea parametrilor

Observatii generale

- parametrii formali - sunt creati la intrarea intr-o functie si distrusi la retur;
- apel prin valoare - copiaza valoarea unui argument intr-un parametru formal ⇒ modificarile parametrului nu au efect asupra argumentului;
- apel prin referinta - in parametru este copiata adresa unui argument ⇒ modificarile parametrului au efect asupra argumentului.
- functiile, cu exceptia celor de tip void, pot fi folosite ca operand in orice expresie valida.

4. Curs 1

4.1 Completări aduse de limbajul C++ faţă de limbajul C

Functii in structuri

C

```
#include <stdio.h>
#include <stdlib.h>
struct test
{
  int x;
  void afis()
  {
    printf("x= %d",x);
  }
}A;

int main()
{
    scanf("%d",&A.x);
    A.afis(); /* error ‘struct test’ has no member called afis() */
    return 0;
}
```

C++

```
#include <iostream>
using namespace std;
struct test
{
  int x;
  void afis()
  {
    cout<<"x= "<<x;
  }
}A;

int main()
{
    cin>>A.x;
    A.afis();
    return 0;
}
```

4. Curs 1

4.1 Completări aduse de limbajul C++ faţă de limbajul C

Functii in structuri

```
#include <stdio.h>
#include <stdlib.h>

struct test {
  int x;
  void (*afis)(struct test *this);
};

void afis_implicit(struct test *this) {
  printf("x= %d",this->x);
}

int main() {
  struct test A = {3, afis_implicit};
  A.afis(&A);
  return 0;
}
```

Q: Exista un mecanism prin care putem avea totusi functii in structuri in C?
A: Da, utilizand pointerii la functii

Q: Codul alaturat este valid si in C++?
A: Nu, pentru ca am folosit “this” ca identificator (mai tarziu despre “this”).

Q: Daca putem folosi, totusi, functii in structuri in C, de ce folosim clase?
A: Pentru ca e dificil de emulat ascunderea informatiei, principiu de baza in POO.

4. Curs 1

4.2 Principiile programarii orientate pe obiecte

Principalele concepte utilizate in POO sunt:

Obiecte

Clase

Mostenire

Ascunderea informatiei

Polimorfism

***Sabloane* – nu sunt utilizate strict POO (mai general, se refera la Programarea generica)**

4. Curs 1

4.2 Principiile programarii orientate pe obiecte

O **clasa** defineste atribute si metode.

```
class X{
        //date membre
        //metode (functii membre – functii cu argument implicit obiectul curent)
        };
```

•mentioneaza proprietatile generale ale obiectelor din clasa respectiva

•folositoare la encapsulare (ascunderea informatiei)

•reutilizare de cod: mostenire

4. Curs 1

4.2 Principiile programarii orientate pe obiecte

Un **obiect** este o instanta a unei clase care are o anumita stare (reprezentata prin valoare) si are un comportament (reprezentat prin functii) la un anumit moment de timp.

- au stare si actiuni (metode/functii)
- au interfata (actiuni) si o parte ascunsa (starea)
- Sunt grupate in clase, obiecte cu aceleasi proprietati

Un **program orientat obiect** este o colectie de obiecte care interactioneaza unul cu celalalt prin mesaje (aplicand o metoda).

4. Curs 1

4.2 Principiile programarii orientate pe obiecte

Principalele concepte (caracteristici) ale POO sunt:

Incapsularea – contopirea datelor cu codul (metode de prelucrare si acces la date) în clase, ducând la o localizare mai bună a erorilor şi la modularizarea problemei de rezolvat;

Moştenirea - posibilitatea de a extinde o clasa prin adaugarea de noi functionalitati;

- multe obiecte au proprietati similare
- reutilizare de cod

4. Curs 1

4.2 Principiile programarii orientate pe obiecte

Principalele concepte (caracteristici) ale POO sunt:

Ascunderea informatiei

foarte importanta

public, protected, private

Avem acces?	**public**	**protected**	**private**
Aceeasi clasa	da	da	da
Clase derivate	da	da	nu
Alte clase	da	nu	nu

4. Curs 1

4.2 Principiile programarii orientate pe obiecte

Principalele concepte (caracteristici) ale POO sunt:

Polimorfismul (la executie – *discutii mai ample mai tarziu*) – într-o ierarhie de clase obtinuta prin mostenire, o metodă poate avea implementari diferite la nivele diferite in acea ierarhie;

4. Curs 1

4.2 Principiile programarii orientate pe obiecte

Alt concept important in POO:

Sabloane

•cod mai sigur/reutilizare de cod

•putem implementa lista inlantuita de

–intregi

–caractere

–float

–obiecte

Perspective

1. Se vor discuta directiile principale ale cursului, feedback-ul studentilor fiind hotarator in acest aspect

- intelegerea notiunilor

- intrebari si sugestii

2. Cursul 2:

- Introducere in OOP. Clase. Obiecte